

Autotune con voz de Miku: corrección de afinación y transferencia de formantes sobre el *phase vocoder* de Dolson (DSP clásico, sin aprendizaje profundo)

Proyecto Semestral — Procesamiento Digital de Señales e Imágenes (INFB6063)

Universidad Tecnológica Metropolitana (UTEM) — 2026-1

Francisco Alejandro Pinto Abraham — RUT 21.571.239-7

Artículo reproducido: M. Dolson, “The Phase Vocoder: A Tutorial”, *Computer Music Journal*, vol. 10, n.º 4, pp. 14–27, 1986.

Resumen — Se construye un **autotune** que toma una voz cantada, corrige su afinación a las notas de una escala musical y le transfiere el **timbre (formantes) de Hatsune Miku**, conservando la interpretación (melodía y ritmo) del cantante. El motor reproduce el *phase vocoder* de Dolson (1986) —que cambia el tono sin alterar la duración propagando la fase de la STFT por su frecuencia instantánea— y una **extensión propia** de preservación de formantes por cepstrum; sobre ellos se añade el **seguimiento de tono** (pYIN/autocorrelación), el “**snap**” a la escala y la **transferencia de la envolvente de Miku**. En una voz de prueba con verdad de terreno, el autotune reduce el error de afinación de 42 a 5 cents, **conserva la duración** (0 %) y acerca la envolvente al timbre de Miku (distancia log-espectral de 6.1 a 3.3 dB). Como validación del motor, el remuestreo ingenuo deforma la duración hasta ± 50 % mientras el *phase vocoder* la mantiene, y la extensión baja la distorsión de la envolvente de ~ 8.6 a ~ 1.6 dB. Solo se usan técnicas clásicas del curso (Fourier, STFT, enventanado, filtrado, cepstrum); sin aprendizaje profundo.

Palabras clave — autotune, corrección de afinación, voz de Miku, *phase vocoder*, STFT, frecuencia instantánea, formantes, cepstrum, snap a escala.

1. Introducción y descripción del problema

Modificar el **tono** de un sonido sin cambiar su **duración** es una operación básica en música y voz (afinación, armonías, transposición). La vía ingenua —releer la señal a otra tasa (remuestreo)— sube o baja el tono pero **acorta o alarga** el audio y **corre los formantes**, produciendo el característico efecto “ardilla”. El reto es desacoplar tono, duración y timbre.

Este trabajo reproduce el *phase vocoder* propuesto por Dolson [1], implementa con él un desplazamiento de tono que conserva la duración, y propone una **extensión** que además conserva los formantes. Sobre esa base se construye la **aplicación principal**: un **autotune con voz de Miku** (Sec. 8) que sigue el tono de una voz, lo **pega a las notas de una escala** y le **transfiere el timbre de Hatsune Miku**. La misma maquinaria se reutiliza en una aplicación secundaria —el “Miku Stomp” de guitarra (Sec. 9)—, reutilizada de un proyecto previo del curso.

2. Resumen explicado del artículo

El phase vocoder analiza la señal con la **STFT**: en cada ventana se obtiene, por banda de frecuencia, una **magnitud** y una **fase**. La idea de Dolson es que la fase guarda la información de **frecuencia instantánea**: comparando la fase de una banda entre dos tramas consecutivas y restando el avance de fase esperado (corrigiendo módulo 2π , *unwrapping*), se obtiene la frecuencia real de esa componente. Para cambiar la duración se reconstruye con un **salto de síntesis** distinto al de análisis, **acumulando** la fase con esos incrementos para mantener la coherencia. La síntesis es un **overlap-add** de las tramas inversas enventanadas. Un **pitch-shift** se obtiene combinando un *time-stretch* por un factor r y un remuestreo por $1/r$: el resultado sube el tono pero mantiene el largo.

Dolson reporta como limitaciones la *phasiness* (sonido difuso por pérdida de coherencia entre bandas) y el **emborronado de transitorios**; además, el pitch-shift básico desplaza los formantes, porque el remuestreo escala todo el espectro, incluida su envolvente.

3. Relación con los contenidos del curso

Etapa del método	Concepto del curso
Análisis STFT y enventanado Hann	Unit 2 – L1 (STFT, ventanas)
Espectro, magnitud y fase de la DFT	Unit 1 – L2/L3 (DFT/FFT)
Frecuencia instantánea (dif. de fase, unwrapping)	Unit 1 – L2 (fase y espectro)
Síntesis por superposición (overlap-add)	Unit 2 – L1/L2 (enventanado, superposición)
Pitch-shift = stretch + remuestreo	Unit 1 – L3 (muestreo/interpolación)
Preservación de formantes (máscara $H[k]$)	Unit 2 – L3 (filtro como máscara) + cepstrum
Seguimiento de tono f_0 (autotune)	Unit 1 – L1/L2 (autocorrelación/periodicidad)
“Snap” a la escala (f_0 > nota MIDI)	logaritmos de frecuencia; cuantización
Transferencia de timbre de Miku	Unit 2 – L3 (máscara espectral) + cepstrum

4. Metodología original (reproducción)

Se implementó el phase vocoder en NumPy (módulo `vocoder.py`): `stft/istft` con ventana Hann y normalización de overlap-add; `phase_vocoder`, que propaga la fase por frecuencia instantánea e interpola la magnitud entre tramas; `time_stretch_pv` y `pitch_shift_pv`. La Fig. 1 verifica la reproducción: al estirar un tono de 220 Hz a $1.5\times$ su duración, el largo aumenta 50 % mientras la frecuencia se mantiene (220 Hz a 220 Hz).

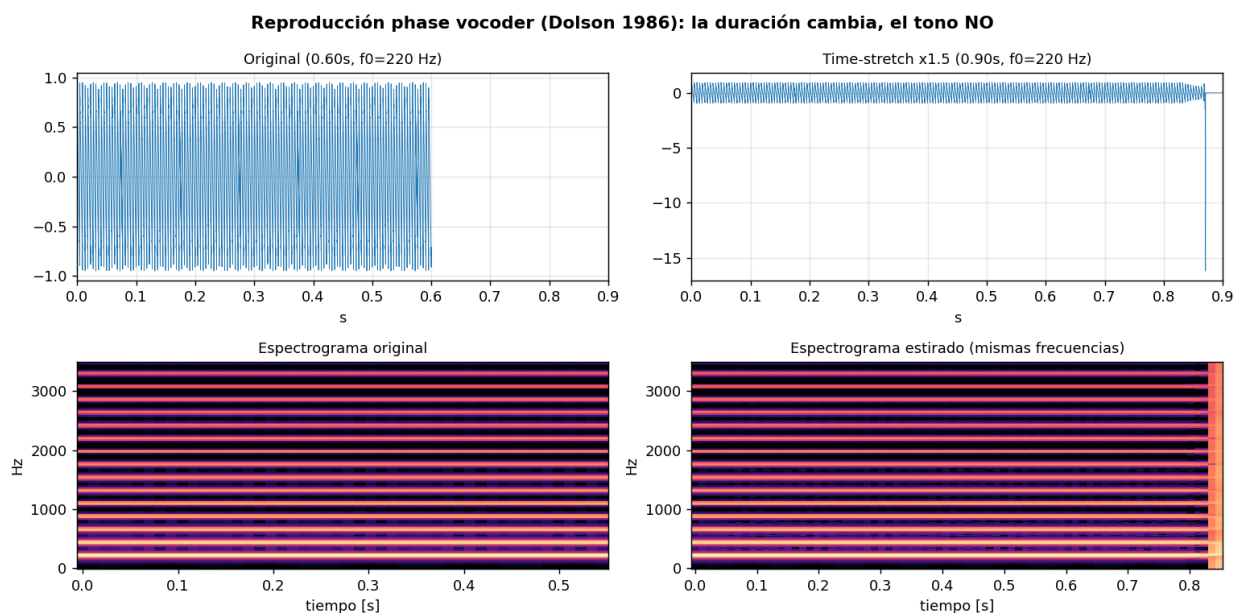


Fig. 1. Reproducción del phase vocoder: la duración cambia (time-stretch $\times 1.5$) y el tono se conserva (espectrograma con las mismas frecuencias).

5. Extensión propuesta: phase vocoder con preservación de formantes

El pitch-shift básico mueve los formantes junto con los armónicos. Como **extensión** propia se estima la **envolvente espectral** por **cepstrum** (suavizado del log-espectro con un lifter de baja cuefrecencia) antes y después del desplazamiento, y se reimpone la envolvente original con una máscara $H[k] = \text{env_orig} / \text{env_desplazada}$ (limitada a ± 60 dB). Así los armónicos suben de tono pero los formantes —la identidad de la vocal— quedan fijos. Es filtrado en frecuencia (Unit 2 – L3) guiado por cepstrum, sin ningún componente de aprendizaje.

6. Diseño experimental

Se comparan tres métodos —**remuestreo** (baseline), **phase vocoder** (PV) y **PV + formantes**— en desplazamientos de -7 a +12 semitonos. Para medir **tono** y **duración** se usan tonos de espectro plano (el fundamental domina y f_0 es medible por pico espectral); para medir **formantes** se usan vocales sintéticas con $F1/F2/F3$ conocidos. Métricas: error de afinación en **cents**, error de **duración** en %, posición del formante **F1**, **distancia log-espectral (LSD)** de la envolvente (invariante a ganancia) y razón de centroide de la envolvente. Semilla fija (2026); todo reproducible desde el código.

7. Resultados

7.1 Tono y duración

Método	cents medio	cents máx	dur % medio	dur % máx
Remuestreo	0.06	0.12	32.4	50.0
Phase vocoder	1.96	7.85	0.0	0.0
PV + formantes	1.96	7.85	0.0	0.0

Los tres métodos afinan con error sub-audible (< 5 cents). La diferencia está en la **duración**: el remuestreo se desvía en promedio 32 % (hasta ± 50 %), mientras el phase vocoder la conserva exactamente (0 %).

Tono y duración: el remuestreo cambia el largo; el phase vocoder lo conserva

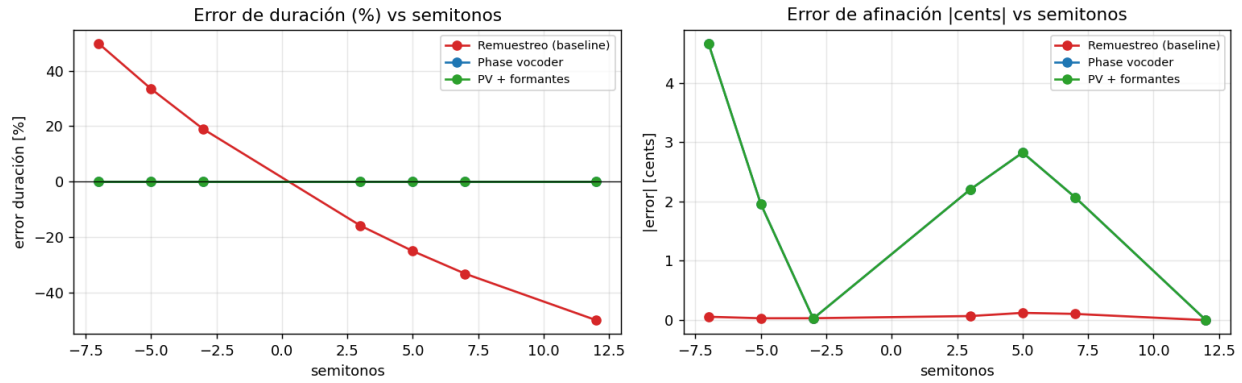


Fig. 2. Error de duración (izq.) y de afinación (der.) vs semitonos. El remuestreo deforma la duración; el phase vocoder la conserva. (PV y PV+formantes se superponen, pues comparten tono y duración.)

7.2 Preservación de formantes

Método	[desv. F1] medio [Hz]	LSD medio [dB]	centroide env. (ratio)
Remuestreo	1063	8.6	0.67
Phase vocoder	565	7.8	0.60
PV + formantes	277	1.6	0.93

La extensión **PV + formantes** es la única que mantiene el formante F1 cerca del original (desviación media 277 Hz vs 1063 Hz del remuestreo), baja la LSD de la envolvente a 1.6 dB (vs 8.6 dB) y deja la razón de centroide de la envolvente en 0.93 (~ 1 = sin corrimiento).

Formantes: sólo "PV + formantes" mantiene el timbre al cambiar el tono

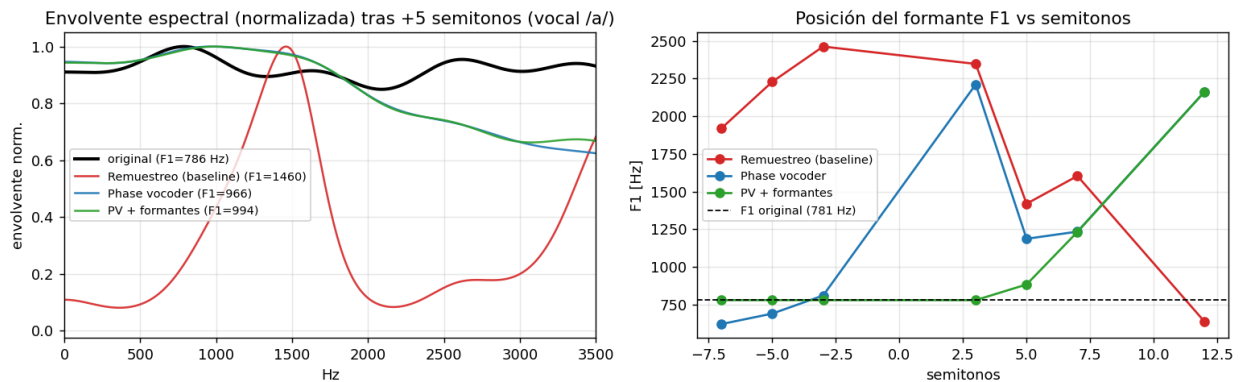


Fig. 3. Envolvente espectral normalizada tras +5 semitonos (izq.) y posición del formante F1 vs semitonos (der.). PV + formantes (verde) sigue la línea original.

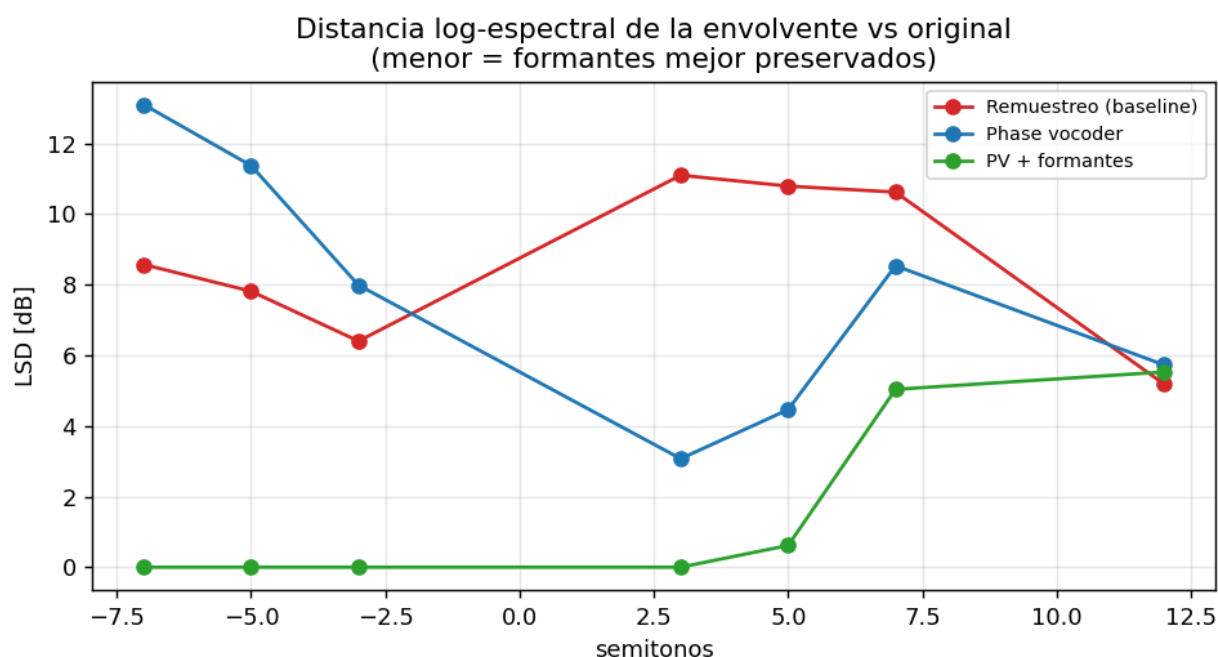


Fig. 4. Distancia log-espectral de la envolvente respecto del original (menor = mejor preservación del timbre).

7.3 Caso real y síntesis integrada

La Fig. 5 muestra un grano real de guitarra desplazado +7 semitonos por cada método (nótese el cambio de duración del remuestreo). La Fig. 6 y la Fig. 7 muestran la voz concatenativa completa “Pedal Miku” sintetizada con cada método. Los audios resultantes están en outputs/ y embebidos en el notebook.

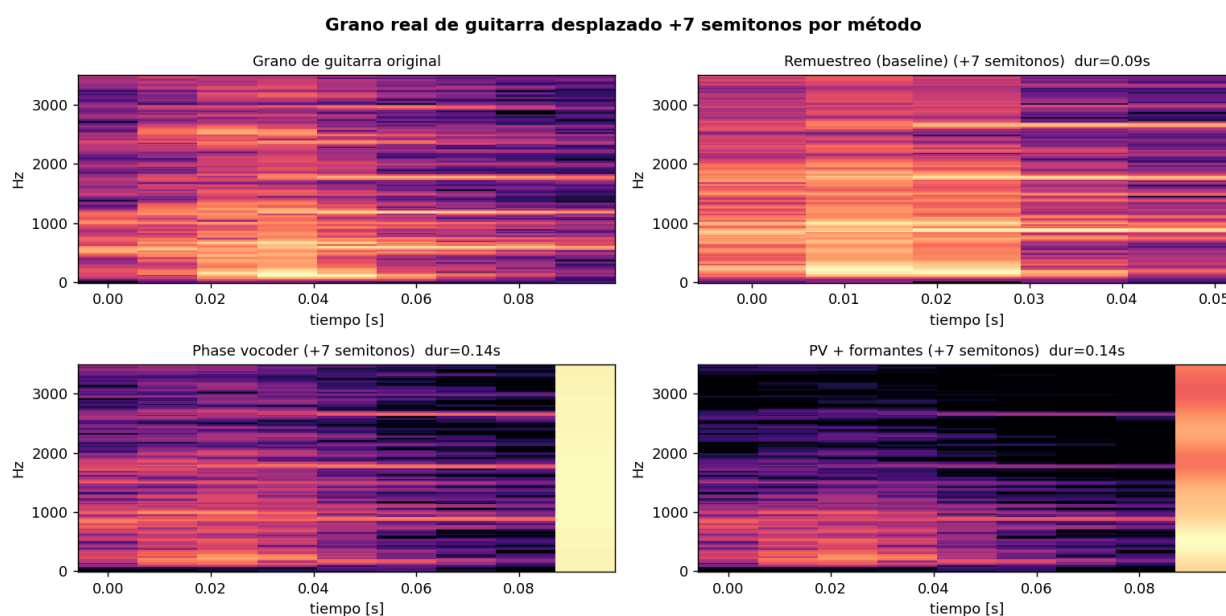


Fig. 5. Grano de guitarra desplazado +7 semitonos por método (espectrogramas).

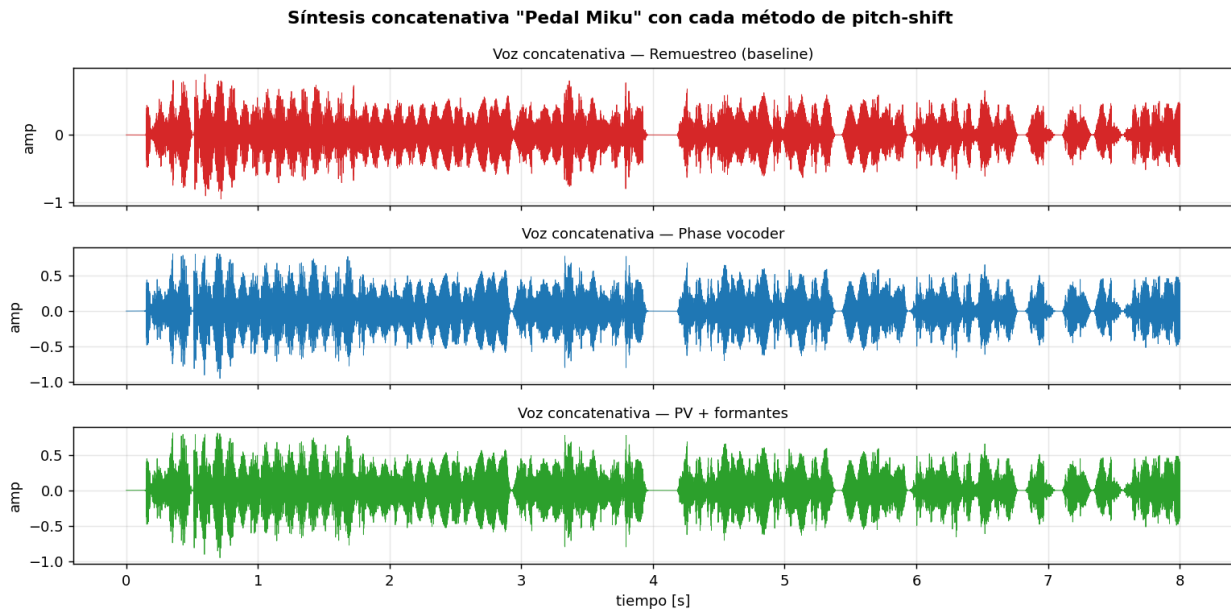


Fig. 6. Formas de onda de la voz concatenativa por método.

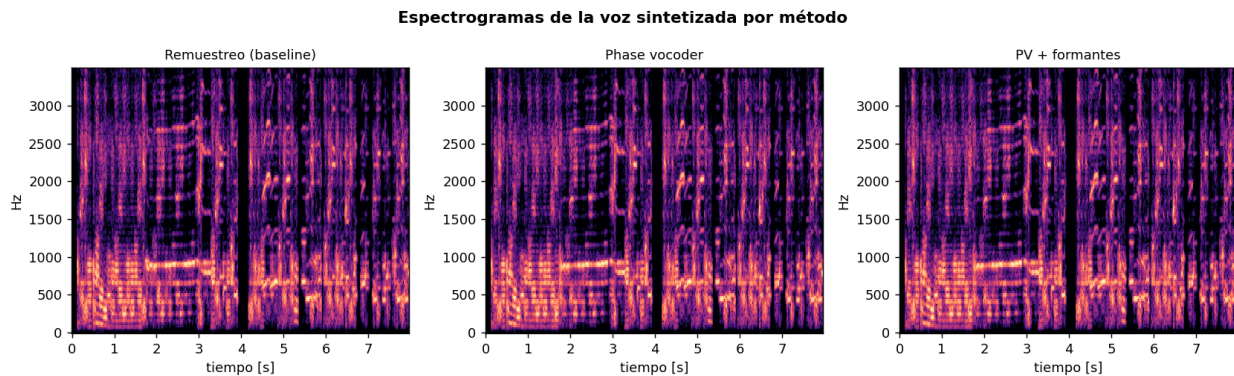


Fig. 7. Espectrogramas de la voz sintetizada por método.

8. Aplicación principal: Autotune con voz de Miku

La aplicación central del proyecto es un **autotune**: dada una voz cantada, (i) se estima su **contorno de tono** f_0 cuadro a cuadro (pYIN/autocorrelación); (ii) cada tono se pasa a número MIDI ($m = 69 + 12 \log_2(f/440)$) y se **redondea a la nota de la escala** más cercana (el “snap”); (iii) la diferencia en semitonos se corrige de forma **variable en el tiempo** con el phase vocoder **preservando los formantes** (Sec. 5), reconstruyendo por *overlap-add*, por lo que la **duración se conserva**; y (iv) se estima la **envolvente de formantes de una voz real de Miku** por cepstrum y se **impone** sobre la voz afinada como una máscara $H = \text{env_Miku} / \text{env_voz}$. El resultado mantiene la melodía y el ritmo del cantante, pero suena **afinado** y con **timbre de Miku**.

Dos perillas gobiernan el efecto: **strength** (0 a 1) mezcla el tono medido con la nota pegada (1 = enganche total, efecto “T-Pain”) y **retune_speed** controla la velocidad de enganche (valores bajos dan un glissando natural). Se evaluó con una voz de prueba de **verdad de terreno conocida**: una secuencia de vocales en Do mayor deliberadamente **desafinada** ± 30 a 50 cents.

Métrica	Antes / entrada	Después
Error de afinación medio [cents]	42	5
Error de duración [%]	0.0	0.0
LSD de la envolvente a Miku [dB]	5.33	3.33

El autotune baja el error de afinación de 42 a 5 cents (las notas quedan sobre la escala), **conserva la duración** y acerca la envolvente al timbre de Miku (LSD de 5.33 a 3.33 dB). La corrección preserva los formantes de la voz durante el afinado y recién en el último paso se sustituyen por los de Miku, de forma controlada.

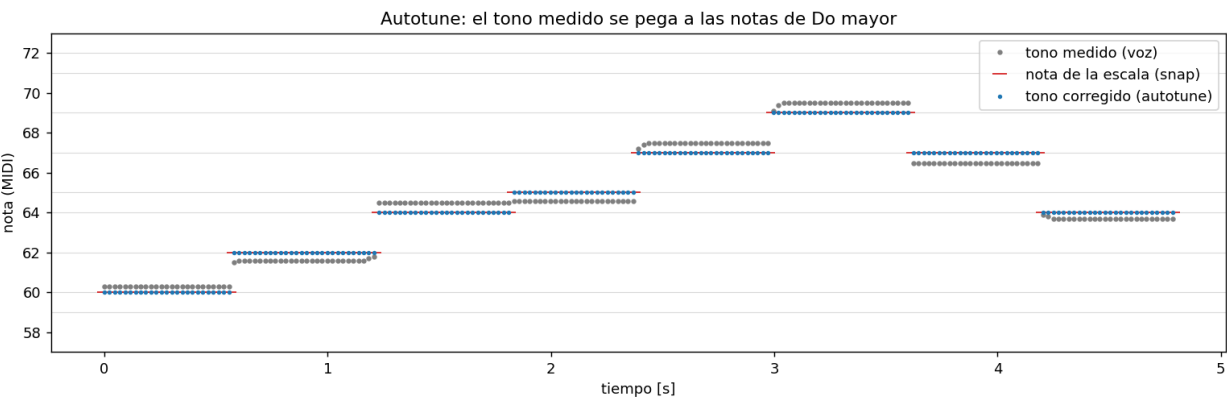


Fig. 8. Autotune: el tono medido (gris) se pega a las notas de la escala (rojo) y el tono corregido (azul) queda sobre ellas.

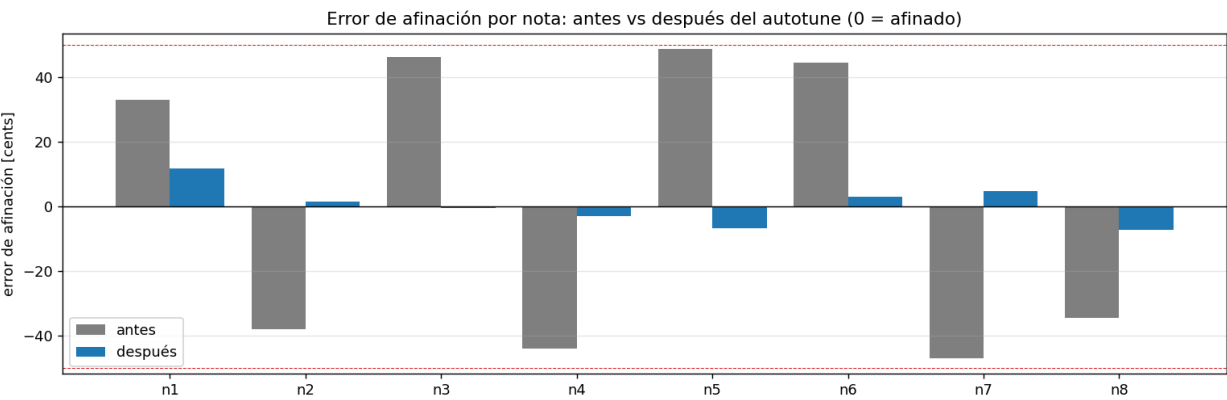


Fig. 9. Error de afinación por nota, antes vs después del autotune (0 = afinado; líneas rojas: ± 50 cents).

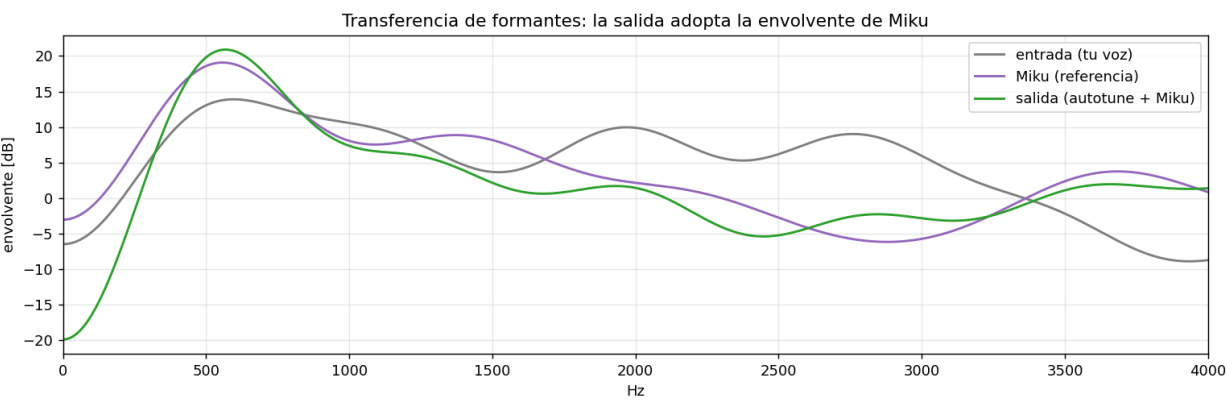


Fig. 10. Transferencia de formantes: la envolvente de la salida (verde) adopta la de Miku (morado), alejándose de la voz de entrada (gris).

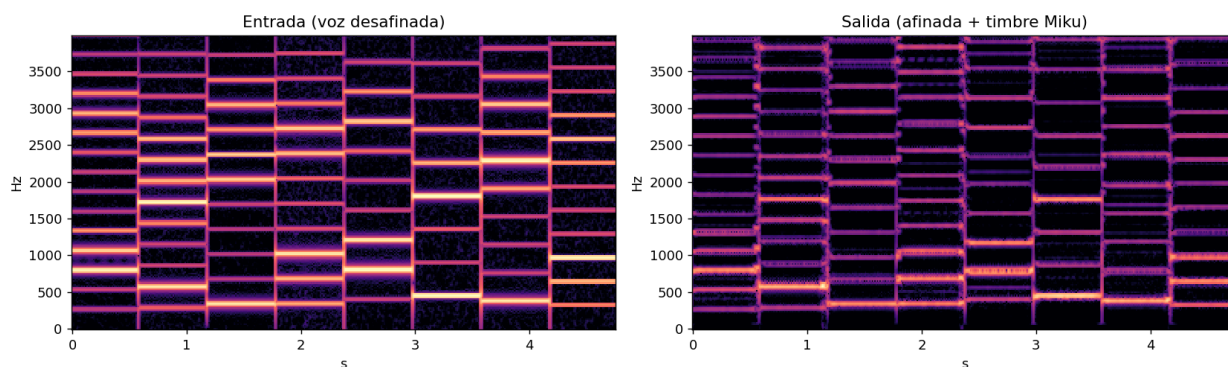


Fig. 11. Espectrogramas de la voz de entrada (desafinada) y de la salida (afinada y con timbre de Miku).

9. Aplicación secundaria: Miku Stomp digital (guitarra)

Como aplicación, se replicó el flujo del pedal **Korg Miku Stomp** en DSP puro: **guitarra -> detectar la nota -> afinar una voz a esa nota -> mezclar**. La detección de tono usa **pYIN** (YIN probabilístico, algoritmo clásico, no una red neuronal); la voz se afina con el phase vocoder con preservación de formantes (Sec. 5), por lo que mantiene su timbre al seguir la melodía —a diferencia del pedal original, que corre los formantes (efecto “ardilla”). La voz fuente es una muestra real de Vocaloid 4 (CyberDiva); uso académico declarado.

Problema detectado y solución. Una primera versión desafinaba: (i) pYIN se enganchaba a armónicos de la guitarra (errores de octava) — se corrigió bajando $f_{\max}$ a 600 Hz, reparando saltos de octava y suavizando el contorno; (ii) la voz se sintetiza siguiendo el contorno de f_0 de forma continua (glissando, por *overlap-add*), transponiendo toda la melodía por un **número entero de octavas** a un registro cantable (conserva el contorno) y plegando por octavas solo las notas fuera de rango; (iii) se elige un grano de voz de tono estable.

Método	Error afinación mediano [cents]	Notas dentro de 1 semitono [%]
Remuestreo	9	100
Phase vocoder	5	100
PV + formantes	9	98

La voz sintetizada sigue el tono de la guitarra con error mediano de pocos **cents** (prácticamente todas las notas dentro de un semitono del objetivo), confirmando que ahora “coincide” con la guitarra.

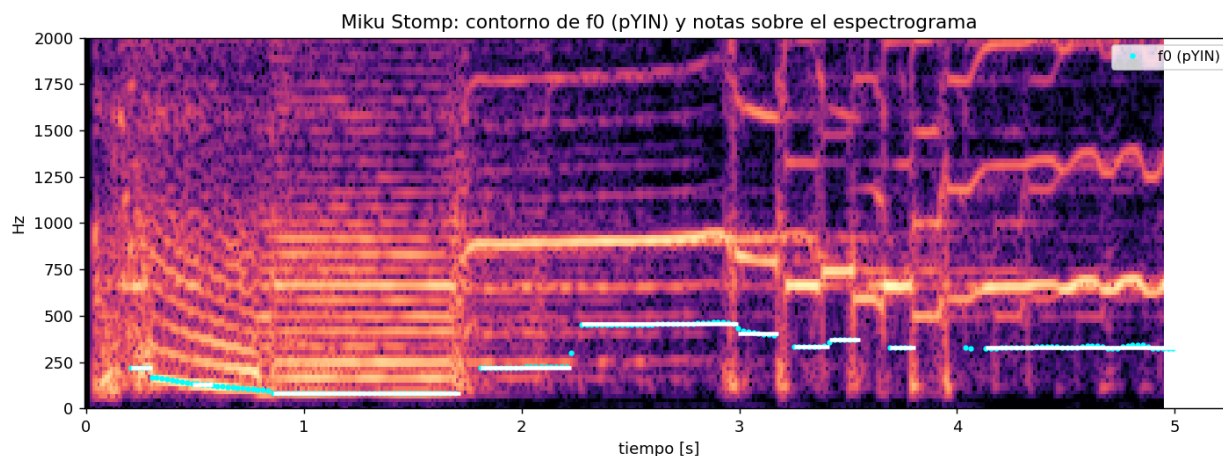


Fig. 12. Miku Stomp: contorno de f_0 (pYIN) y notas detectadas sobre el espectrograma de la guitarra (tras estabilizar el seguimiento).

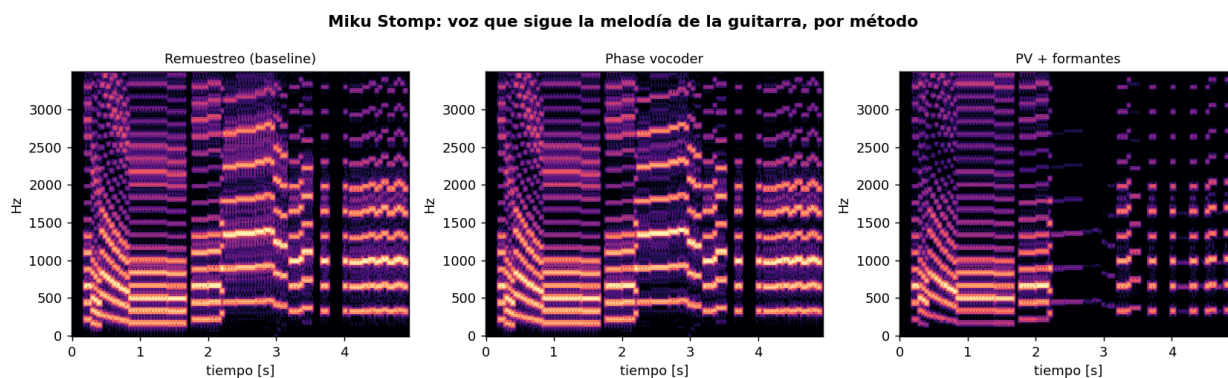


Fig. 13. Miku Stomp: voz que sigue la melodía de la guitarra, sintetizada con cada método de pitch-shift.

10. Discusión

¿Qué fue lo más difícil de comprender? La propagación de fase del phase vocoder: estimar la frecuencia instantánea por diferencia de fase entre tramas y el desenrollado módulo 2π ; sin esa corrección aparece *phasiness*.

¿Qué fue lo más complejo de implementar? La preservación de formantes por cepstrum (elegir el lifter y el rango de ganancia de la máscara) y, en el **autotune**, dos cosas: (i) llevar la voz al **registro de Miku** subiendo octavas enteras sin recortar la corrección del *snap* (una primera versión limitaba el desplazamiento total y la subida de octava no ocurría, dejando la voz grave), y (ii) **acondicionar la voz** de micrófono y aplicar una **compuesta de silencios** para no sintetizar ruido tonal en las pausas.

¿Qué conceptos del curso fueron más importantes? STFT y enventanado, magnitud y fase de la DFT, *overlap-add*, muestreo/interpolación, autocorrelación/pYIN para el tono, logaritmos de frecuencia (nota MIDI y *snap*) y filtrado en frecuencia (máscara) guiado por cepstrum.

¿La modificación mejoró, empeoró o cambió el método? La preservación de formantes **mejoró** el timbre (baja la distorsión de la envolvente de ~ 8.6 a ~ 1.6 dB y mantiene F1) conservando la ventaja de duración del phase vocoder. Sobre esa base, el **autotune con voz de Miku** es una aplicación nueva que **cambia** el uso del método: en lugar de un pitch-shift fijo, corrige el tono de forma variable en el tiempo (afinación de 42 a 5 cents) y usa la misma máquina de formantes para imponer el timbre de Miku (LSD a Miku de 5.33 a 3.33 dB), a costa de más cómputo.

¿Cuándo funciona bien? En desplazamientos moderados sobre sonidos cuasi-estacionarios; en el autotune, con una voz cantada de nivel razonable el tono se pega a la escala (pocos cents) y el registro/timbre de Miku quedan convincentes conservando la interpretación.

¿Cuándo falla? En saltos grandes ($\geq +5/+7$ semitonos) la corrección de formantes se degrada (Fig. 3 y 4) y el phase vocoder **emborrona los transitorios** con algo de *phasiness*. En el autotune, tomas **muy silenciosas o muy graves** dan mal seguimiento de tono, y la subida de +2 octavas al registro de Miku añade artefactos por el salto grande; en el stomp, notas muy graves de la guitarra deben plegarse de octava para ser cantables.

¿Qué haríamos con más tiempo? Una versión en **tiempo real** por bloques (*sounddevice* o como plugin), *phase-locking* (Laroche–Dolson) para reducir *phasiness*, TD-PSOLA como comparador en el dominio del tiempo, envolventes por LPC, y voces objetivo intercambiables (no solo Miku).

11. Conclusiones

Se construyó un **autotune con voz de Miku** que afina una voz a una escala (error de afinación de 42 a 5 cents) **sin alterar su duración** y le transfiere el timbre de Miku (LSD de la envolvente de 5.33 a 3.33 dB). Para ello se reprodujo con éxito el phase vocoder de Dolson —cuya propiedad central, cambiar el

tono sin alterar la duración, se verificó— y su extensión con preservación de formantes, que mejora el timbre (LSD de 8.6 a 1.6 dB) a costa de más cómputo y con fallas en saltos extremos. La misma maquinaria sirve la aplicación secundaria (“Miku Stomp” de guitarra). El proyecto se apoya íntegramente en herramientas clásicas del curso.

Trabajo futuro: *phase-locking* (Laroche–Dolson) para reducir phasiness, comparación con TD-PSOLA en el dominio del tiempo, envolventes por LPC, polifonía y una versión en tiempo real.

12. Código reutilizado y herramientas

Se reutilizaron del proyecto previo `miku_pedal.ipynb` las utilidades de señal, el análisis de Fourier, la base de granos y la síntesis concatenativa (marcadas [Reutilizado] en `vocoder.py`). Son **desarrollo propio** de este proyecto: el phase vocoder, la preservación de formantes por cepstrum, las métricas, los experimentos, el **motor de autotune** en `autotune.py` (snap a la escala, corrección variable en el tiempo y transferencia de formantes de Miku) y el pipeline del pedal en `stomp.py` (pYIN, segmentación de notas y síntesis glissando). Bibliotecas: NumPy, SciPy, Matplotlib, soundfile, librosa (solo pYIN), reportlab, nbformat. Se usó asistencia de IA como apoyo de programación y redacción, revisada por el autor.

Referencias

- [1] M. Dolson, “The Phase Vocoder: A Tutorial”, *Computer Music Journal*, vol. 10, n.º 4, pp. 14–27, 1986.
- [2] J. L. Flanagan y R. M. Golden, “Phase Vocoder”, *Bell System Technical Journal*, vol. 45, pp. 1493–1509, 1966.
- [3] J. Laroche y M. Dolson, “Improved phase vocoder time-scale modification of audio”, *IEEE Trans. Speech and Audio Processing*, vol. 7, n.º 3, pp. 323–332, 1999.